

CS 2A Final Exam

- Due Mar 27 at 9pm
- Points 40
- Questions 40
- Available Mar 27 at 6pm - Mar 27 at 9pm 3 hours
- Time Limit 120 Minutes

Instructions

Once you begin, you will have 120 minutes to complete. If you do not submit before that time your incomplete exam will be automatically submitted as is.

You can look at lectures or texts, or use IDEs, but you may not consult any other individuals for help.

Each of the 40 questions is worth 1 point, for an exam total of 40 points.

Multiple choice questions with square check-boxes may have more than one correct answer. Multiple choice questions with round radio-buttons have only one correct answer.

Any code fragments you are asked to analyze are assumed to be contained in a program that has all the necessary variables defined and/or assigned.

Ignore programming style related questions. I'll reinstate any points these questions are worth. I think I've removed them all from the question banks.

Finally, as always, read any code in the questions carefully.

Good luck!

&

Attempt History

	Attempt	Time	Score
LATEST	Attempt 1	99 minutes	35.33 out of 40

❗ Correct answers are hidden.

Score for this quiz: 35.33 out of 40

Submitted Mar 27 at 8:09pm

This attempt took 99 minutes.



Question 1

1 / 1 pts

Not counting consts, it is normally preferred to pass information to a method by using "global variables" rather than by passing arguments to its formal parameter list..

- ☐ True
- ☒ False

Feedback We avoid using globals or class variables to pass information between methods because of the large risk of error and exposure. In C++ globals are highly discouraged.



Question 2

1 / 1 pts

What is the value of the variable result after the following statement has been executed?

```
int result = 1+2 * 3+4;
```

- ☐ 21
- ☐ 13
- ☐ 15
- ☒ 11



Question 3

1 / 1 pts

Consider **carefully** the program fragment below:

```
int a = 3;
...
if (a/2 == 1.5)
    cout << "a = 3\n";
else
    cout << "a != 3\n";
```

How can you correct the error in the program above? (More than one option may be correct)

- ☐ By replacing `a/2` with `a`
- ☒ By replacing `a` in the numerator with `(double) a` or `(float) a`
- ☐ By replacing `==` with `!=`
- ☒ By replacing the 2 in the denominator of `a/2` with `2.f`, `2.` or `2.0`



Question 4

1 / 1 pts

What does the statement

```
a /= 2;
```

do?

- ☒ It replaces the value of `a` by half its value
- ☐ The statement tests whether the value of `a` is not equal to 2
- ☐ It returns half of `a` while leaving `a` unchanged
- ☐ The statement is syntactically incorrect (will cause a compiler error)



Question 5

1 / 1 pts

In the following statement:

```
foo = 'C' + 1;
```

Which of the following is an appropriate choice of data type of the variable `foo`?

(More than one may be correct)

- ☒ `char`
- ☐ `string`
- ☐ None of the other choices since the statement is syntactically incorrect regardless of its data type.
- ☒ `int`



Question 6

1 / 1 pts

Consider **carefully** the following program fragment:

```
for (int i = 0; i < 100; i++) {  
    if (i % 4 == 0 && i % 5 == 0)  
        continue;  
    if (i == 60)  
        break;  
    cout << i << endl;  
}
```

What will its output be?

PS. Consider 0 to be a multiple of every number.

- ☐ A list of all numbers starting at 0 up to 59, except multiples of 4 or 5
- ☐ A list of all numbers starting at 0 up to 99, except multiples of 4 or 5
- ☐ A list of all numbers starting at 0 up to 59, except multiples of 20
- ☒ A list of all numbers starting at 0 up to 100, except multiples of 20



Question 7

1 / 1 pts

Consider **carefully** the program fragment below:

```
int sum = 0, i = 0;  
while (++i < 5) {  
    sum = sum + i;  
}
```

What is the value of the variable `i` **after** the loop exits?

- ☐ 6
- ☐ 4
- ☐ undefined

5



Question 8

1 / 1 pts

In a statement that has multiple "else if" clauses, each "else if" should always be indented to the right of the one above it (i.e., *progressive indentation* should be used.)

- ☒ False
- ☐ True

Feedback There are many times when all the "else if" clauses should be aligned with each other and not indented. Examples have been given in the lesson on the if/else statement.



Question 9

1 / 1 pts

Assuming `num1` is an **int** and `answer` is a **double** variable, which of the following expressions will always give an accurate, double-precision, answer?

(Check all combinations that give double precision accuracy - there will be more than one that should be checked.)

☒ `answer = (num1 + 1) / 100. ;`

☒ `answer = ( ( (double)num1 + 1 ) / 100 );`

☒ `answer = (double) ( (num1 + 1.) / 100 );`

☐ `answer = (double) ( (num1 + 1) / 100 );`

☐ `answer = (num1 + 1) / 100 ;`

Feedback

If the quotient is computed using int arithmetic BEFORE casting to double, the fractional data is lost, so the answer will, in general, be wrong.

If either the numerator or the denominator is a double before the quotient is computed, then the answer will be promoted to double and no information lost.

Finally, in the case where either of the terms in the numerator sum is cast to double, then that sum will be promoted to double and the entire numerator will become double, which means the quotient will use double arithmetic.



Question 10

1 / 1 pts

Consider the following method definition:

```
int update( int userIn )  
{  
    userIn++;  
    return userIn;  
}
```

Which of the following client calls will always result in the variable **clientVal** being incremented by exactly 1?

- ☐ `update( clientVal );`
- ☒ `clientVal = update( clientVal );`
- ☐ `y = update( clientVal );`
- ☐ `clientVal = update( y );`

Feedback

Since the client argument passed into the method sets the value of the *local variable*, **userIn**, which is *distinct* from that client variable, the method statement **userIn++** *does not affect* the client argument directly. It is only the **return value** that affects the client value, and that happens only if the client variable "catches" the result as a functional return. Since we are asking that the method increment the client variable **clientVal**, we have to both

1. pass **clientVal** to the method, and
2. capture the return into that same variable, **clientVal**

in order to get a reproducible "increment" out of the method.



Question 11

1 / 1 pts

Here is an expression which contains the + operator in several places on the ***RHS*** of an assignment statement (and we don't need to know anything about the shaded variable on the ***LHS*** to answer the question):

```
someVar    = '1' + '2' + '3' ;
```

In this expression ...

- ☐ ... the + operator represents string concatenation.
- ☒ ... the + operator represents addition.

Feedback

The operands are all **chars**, and **chars** are small **ints**, not **strings**. The + operator will add the ASCII values, not concatenate the individual digits.



Question 12

1 / 1 pts

Here is a code fragment which appears in a valid **main()**:

```
int numGifts;
double costPerGift;

// numGifts gets a value somehow in code that is not shown.  then ...

if ( numGifts < -10 )
    costPerGift = 3.0;
else if ( numGifts < 8.5 )
    costPerGift = 2.75;
else
    costPerGift = 2.5;
```

For which values of **numGifts** will **costPerGift** be **2.75** at the end of the fragment?

(Check all boxes that apply - there will be more than one.)

- ☒ numGifts = -1
- ☒ numGifts = 8
- ☐ numGifts = 20
- ☐ numGifts = 15
- ☐ numGifts = 30
- ☐ numGifts = 10



Question 13

1 / 1 pts

Consider the statement:

```
if ( (ans == 'Y' || errors < 5) && numTries < 10 ) // note uppercase 'Y'
    count++;
```

Which combinations of values results in **count** being incremented after the statement is complete?

(Select *all* that apply.)

- ☐ ans = 'N'
errors = 5
numTries = 5
- ☐ ans = 'N'
errors = 3
numTries = 10
- ☐ ans = 'y' (lower case)
errors = 4
numTries = 5
- ☒ ans = 'Y' (upper case)
errors = 6
numTries = 5
- ☒ ans = 'Y' (upper case)
errors = 100
numTries = -1



Question 14

1 / 1 pts

The source code that a programmer writes which describes the exact details of that method's implementation is called the "**method definition**".

- ☒ True
- ☐ False

Feedback

The code that contains the details of how a method accomplishes its goal is called the **method definition**. The method **invocation** (or method **call**) is used by a client or `main()` to get the method to run, but it gives us no information about the exact implementation of the method. The **method prototype**, gives the signature of the method, but it, also, tells us nothing about the method's implementation details.



Question 15

1 / 1 pts

At the **end of the third (3rd)** loop pass of the following loop:

```
for (k = 3, count = -1; k < 10; k++)  
    count++;
```

what is the value of count?

- ☐ 5
- ☒ 2
- ☐ 3
- ☐ 9

Hints

You can "play computer" as you learned to do in week 2 to get the answer.

Feedback

This is not about the final value of `count`, but the value only part way through the loop, an important intermediate stage to be able to predict.



Question 16

1 / 1 pts

It is normally preferred to pass information to a method by passing arguments to its formal parameter list rather than by using "global variables" for this purpose.

- ☐ False
- ☒ True

Feedback We avoid using globals or class variables to pass information between methods because of the large risk of error and exposure. In C++ globals are highly discouraged.



Question 17

1 / 1 pts

Consider the code fragment (assumed to be in a program in which all variables are legally defined):

```
int num1;
double num2;
double answer;

// program gets num1 and num2 from user, and values received
// are always non-zero numbers between +1 and +100 (code not shown)

...

// compute precise quotient:
answer = (double) ( num1 / num2 );
```

After the assignment statement the variable `answer`, will hold the *most precise quotient possible*, accurate to several digits to the right of the decimal point ...

- ☐ ... NEVER, regardless of what values `num1` and `num 2` hold.
- ☒ ... ALWAYS, regardless of what values `num1` and `num 2` hold (as long as they are in the stated range).
- ☐ ... SOMETIMES, i.e., for some, but not all, values of `num1` and `num 2.`

Feedback

If **num1** = 1 and **num2** = 3, then **answer** will be 0.33333 accurate to several decimal places. This is true of **any** two values for the numbers because the first quotient **num1 / num2** is done using **double arithmetic**, *always*. It is a binary (two-number) mixed-mode operation (one int and one double) which will promote the int to a double.



Question 18

1 / 1 pts

Here is an prototype for a working class **OldCellPhone** that has no errors (*you may see this exact class several times in this exam*):

```
class OldCellPhone
{
public:
    static const int MIN_CAP = 10;
    static const int MAX_CAP = 1000;
    static const string DEFAULT_DSCR;

private:
    string description;
    int memCapacity;
    bool camera;
    bool gps;
public:
    OldCellPhone(string dscr = DEFAULT_DSCR, int mem = MIN_CAP,
        bool cam = false, bool gp = false);
    string toString();
    bool setMemCapacity(int mem);
    void setCamera( bool hasCam ) { camera = hasCam; }
    void setGps( bool hasGps ) { gps = hasGps; }
};
```

A programmer decides to add a method **add20ToMemory()** whose job it will be to increase the private int **memCapacity** by **20** as long as the new, increased memory capacity, after adding 20, is still **<= MAX_CAP**. Otherwise, it will not touch **memCapacity**.

Check the true statements (there may be more than one correct answer):

- ☒ If the programmer chose to make it an *instance method*, it would not need any parameters to do its job.
- ☒ This should be an instance method.
- ☐ The method should be a **void** return type.

☒ The method should have a bool return type.

☐ This should be a static method.

If the program chose to make it a **static method**, it would not need any parameters to do its job.

☐

Hints

Feedback

The method signature should be **bool add20ToMemory()** and it should be a **public instance method**, as it modifies private data and has a chance to detect an error (no room to add more memory) so must return a **bool**. If a programmer unwisely tried to make it a **static method**, then it would require an object parameter since there is no object whose **memCapacity** it would be able to increment by 20 without such a parameter.



Question 19

1 / 1 pts

Here is an prototype for a working class **OldCellPhone** that has no errors (*you may see this exact class several times in this exam*):

```
class OldCellPhone
{
public:
    static const int MIN_CAP = 10;
    static const int MAX_CAP = 1000;
    static const string DEFAULT_DSCR;

private:
    string description;
    int memCapacity;
    bool camera;
    bool gps;

public:
    OldCellPhone(string dscr = DEFAULT_DSCR, int mem = MIN_CAP,
                  bool cam = false, bool gp = false);
    string toString();
    bool setMemCapacity(int mem);
    void setCamera( bool hasCam ) { camera = hasCam; }
    void setGps( bool hasGps ) { gps = hasGps; }
};
```

A programmer decides to add a method **addMemory()** whose job it will be to increase the private int **memCapacity** by an amount specified by the client as long as the new, increased memory capacity after increasing it is still **<= MAX_CAP**. Otherwise, it will not touch **memCapacity**.

Check the true statements (there may be more than one correct answer):

- ☐ This should be a **static method**.
- ☒ This should be an **instance method**.
- ☒

If correctly defined, a client could add 30 units of memory to the **OldCellPhone** object, **myCell**, by using the syntax:

```
myCell.addMemory( 30 )
```



If correctly defined, a client could add 30 units of memory to the **OldCellPhone** object, **myCell**, by using the syntax:

```
OldCellPhone::addMemory( myCell(30) );
```



If correctly defined, a client could add 30 units of memory to the **OldCellPhone** object, **myCell**, by using the syntax:

```
OldCellPhone::addMemory( 30 );
```

Feedback

The method signature should be **bool addMemory(int memToAdd)** and it should be a **public instance method**, as it modifies private data and has a chance to detect an error (no room to add more memory) so must return a **bool**. The only way a client could modify the object **myCell** using this method would be to use **myCell.addMemory(30);** Other attempts either have inadequate information or are unreasonable and invalid calls no matter how **addMemory ()** was designed.



Question 20

1 / 1 pts

Consider this method definition:

```
int someRecMethod( int n )
{
    if ( n < 0 )
        return -1;
    return someRecMethod( n + 1 );
}
```

This method has the following problem -- or none (only one correct choice):

☐ Nothing is wrong; it has both an explicit case and a recursive case.



It will sometimes return a -1 without error, and other times produce a runaway recursive call to itself resulting in a run-time error.

☐ It will always return the same number no matter what is passed in as an argument from the client.

☐ It will always produce a runaway recursive call to itself resulting in a run-time error.

Feedback

The recursive call does not move the parameter it passes to itself closer to the explicit "escape" case of $n < 0$. Instead, if a positive number is passed to it, it creates a number even greater ($n+1$) before it makes the recursive call making the new argument even farther from the necessary explicit condition of ($n < 0$) needed to end the recursion. If the client passes in a negative number, all is well, and it returns -1. But if the client passes in a 0 or positive int, there will be an infinite recursive call, which will create a run-time error.



Question 21

1 / 1 pts

Which are true about instance variables of a class (check *all* that apply):



Loop counters and other helper variables should be declared as instance variables so that such counters and helpers don't have to be re-declared local to every member method, individually.

☐ Member instance methods of the same class must use `this->` before their names in order to access them.

☒ Mutators should protect them by filtering bad parameter values before assigning those values to them.

☒ They should usually be declared `private`.

☐ Member instance methods of the same class must use mutator methods to modify their values.

Feedback

Instance variables should normally be private and protected by mutator methods. Helper variables like loop counters that have no intrinsic meaning to the class, should not be declared as instance variables, but should be local to the methods in which they are used. Instance methods do not need to use mutators or accessors to get to these private data since these methods automatically have direct access to them. Instance methods do not need to use **this->** in order to access them.



Question 22

1 / 1 pts

Here is a method that returns a random **String** (in the form of **symbolic constants** defined elsewhere):

```
string randString()
{
    int testNum;

    testNum = rand() % 5000;

    if ( testNum > 4600 )
        return BAR;
    else if ( testNum > 3000 )
        return CHERRIES;
    else if ( testNum > 1200 )
        return SPACE;
    else
        return SEVEN;
}
```

If this method were called from a program many times, the following lists the **strings** that we would expect it to return, ordered from the **most frequently** returned **strings** (**top** of list) to **least frequently** returned **strings** (**bottom** of list),

(Check the one that fits this description.)

- BAR
- SEVEN
- CHERRIES
- ☐ SPACE

- SPACE
- CHERRIES
- BAR
- ☐ SEVEN
- BAR
- CHERRIES
- SPACE
- ☐ SEVEN
- SEVEN
- BAR
- CHERRIES
- SPACE
- ☐
- SPACE
- CHERRIES
- SEVEN
- ☒ BAR
- SEVEN
- SPACE
- CHERRIES
- ☐ BAR

Feedback

BAR is selected in a range that spans 400

CHERRIES is selected in a range that spans 1600

SPACE is selected in a range that spans 1800

SEVEN is selected in a range that spans 1200

Therefore, when listed from the largest span to the smallest, we get the answer.



Question 23

1 / 1 pts

A **binary search algorithm** is written (as in the modules, for example) which searches a **pre-sorted** array for some user-defined value, **clientData**. If **clientData** is stored in the array, it returns its array position, and if not found, it returns -1 (again, just like in the modules). Assume the array to be searched has 100 data elements in it. (Check *all* that apply):

[NOTE: due to common off-by-one interpretations when counting such things, if your predicted answer is within one (+1 or -1) of a posted option below, you can assume your prediction and the choice you are looking at are equivalent and check that option.]

- ☐ It **may require as many as 99** comparisons of **data** before it returns.
- ☒ It **might** return to the client with an answer after **only one** comparison of **data**.
- ☒ It **will always** return with an answer **in 7 or fewer** comparisons of **data**.
- ☐ It **will always** return with an answer **in 3 or fewer** comparisons of **data**.

Hints

Each recursive call throws away **half of the elements** it gets. The method gets 100 elements, but if **clientData** is not found in position 49 (**first comparison**) the method throws 51 of those elements away and calls itself. This inner call now has 49 elements to search. It tests element 24 against **clientData (second comparison)**, and if it is not a match, throws about away half again, leaving 24. The **next call/comparison**, if needed, throws away about 12. The **next call**, if needed, throws away 6. The **next call** throws away 3. The **next call** throws away 2. Finally we only have **one element to test. Each recursive call does one comparison**. How many comparisons have we done, give or take one, (count above) in the worst case, assuming we have to go all the way down until we only have one element before we find the data we are searching for?

Feedback

The data may be in middle position, position 49, in which case the algorithm only requires one comparison (of data), so it may return after about one comparison. At the other extreme, the data may be in position 0 or 99, or not in the array at all. In those cases it will require testing the middle element about seven times -- as the hint indicates by walking you through the seven tests -- but never more. So seven is the most it would require (+/- 1 to account for off-by-one details). It certainly could easily be more than three (as our example in the modules shows.)



Question 24

1 / 1 pts

Here is a class prototype and some static members, followed by a static initializer:

```
class Student
{
public:
    static const int SORT_BY_FIRST = 88;
```

```
static const int SORT_BY_LAST = 98;
static const int SORT_BY_POINTS = 108;

private:
    static int sortKey;

    // ... etc.
};

int Student::sortKey = 88;
```

Check the true statements (there may be more than one):



If a programmer changes the **first three const** initializations from **88, 98, 108** to **0, 1, 2**, but makes no other change, the program will have a **logic error**.



The initial **sortKey** is **SORT_BY_FIRST**



The initial **sortKey** is **SORT_BY_POINTS**



The initial **sortKey** is **SORT_BY_LAST**



Assuming everything else is correct, a program using the code above will run perfectly but is not making maximum use of the three symbolic constants defined.



If a programmer changes the **first three const** initializations from **88, 98, 108** to **0, 1, 2**, but makes no other change, the program will have a **compiler error**.



The program will not run correctly because a literal is used to initialize **sortKey**.

Feedback

The program has no compiler errors and will run perfectly as is. However, because the literal **88** is used to initialize **sortKey**, it is not capable of handling a change if the programmer modifies the definitions of the constants to 0, 1 or 2 and makes no other change. This will cause a **logical error** (**sortKey** will be initialized to **88**, but **88** no longer means anything), but no compiler error.



PartialQuestion 25

0.33 / 1 pts

A mutator's job is to (check *all* that apply):

- ☒ set values of private data as requested by the client, if the values are within acceptable bounds.
- ☐ protect private data.
- ☐ return the value of private data to the client.
- ☒ print errors if bad data is passed.
- ☒

return some indication of a possible error to the client, if the client requests something of the mutator that it cannot do, based on the state of the object and/or argument passed into the mutator.

Feedback

Mutators are meant to protect private data by testing values passed in before setting the members. They should never print anything, and do return something to the client indicating success or failure. The error can be of any type, like a bad parameter or a full array, not allowing any further data to be added.



IncorrectQuestion 26

0 / 1 pts

ClassB and **ClassA** have no special relationship; they are two separate classes both used by the same **main()**. **void methodB(ClassA aObj)** is a method of **classB**. It takes a **ClassA** object as an ordinary parameter. The client, **main()**, calls **methodB()**.

Here is an outline of the situation:

```
class ClassA
{
private:
    char someMemB;
};

class ClassB
{
public:
    void methodB( ClassA aObj );
};

int main()
{
    ClassA myObjA;
    ClassB myObjB;
```

```
myObjB.methodB ( myObjA );
}
```

Check all the true statements (Check *all* that apply):



methodB() can read from (access) and/or write to (change) **aObj**'s **someMemb** indirectly through appropriate public **ClassA** methods, if **ClassA** offers such methods.



methodB() can read (access) **aObj**'s **someMemb** value directly without changing it, as in **something = aObj.someMemb**.



If **methodB()** modifies **aObj**'s **someMemb**, the client's **myObjA**'s **someMemb** will also be modified, accordingly, when the method returns.



methodB() can change **aObj**'s **someMemb** value through direct assignment, as in **aObj.someMemb = something**.



If **methodB()** modifies **aObj**'s **someMemb**, the client's **myObjA**'s **someMemb** will not be modified when the method returns.

Feedback

Private data of **ClassA** is never accessible from a non-**ClassA** method like **methodB()**, directly, either for read or write. However, mutators and accessors that **ClassA** offers, can always be used for this purpose. Since **myObj** is passed by value to **methodB()**, the local object, **aObj**, inside that method has no effect on the client **myObj**.



Question 27

1 / 1 pts

If two **int** pointers, **pMyAge** and **pYourAge**, both point to the **same int**, **age**, then the statement:

```
age = 7;
```

will have what impact if followed immediately by the following statements? (Only one correct choice).

It will:

cause **both**

```
cout << *pYourAge;
```

and

```
cout << *pMyAge;
```

☒ to display 7.

cause **both**

```
cout << pYourAge;
```

and

```
cout << pMyAge;
```

☐ to display 7.

cause **only**

```
cout << *pMyAge;
```

to display 7, but **not**

☐

```
cout << *pYourAge;
```

cause **only**

```
cout << *pYourAge;
```

to display 7, but **not**

☐

```
cout << *pMyAge;
```

Feedback

There is only one int, and when we change that, the pointers which reference that int though the * operator will see the same value.



Question 28

1 / 1 pts

An **array of 100 elements** is to be sorted using the **bubble sort** in the modules (the one that tests the return value of **floatLargestToTop()** to see if it can return "early".) Check all the true statements about the sort algorithm, i.e., the sort method and its support methods. (Check *all* that apply.)

- ☒ It will **sometimes** return (completely sorted) after only 99 data comparisons.
- ☒ It will always require at least 99 comparisons
- ☐ It will **always** return (completely sorted) after 99 data comparisons.
- ☐ It will **always** require at least one swap.

Feedback

The **floatLargestToTop()** method will compare neighboring elements in a loop. In the first call, the loop goes up to 99: it compares array elements 0 with 1, then 1 with 2, then 2 with 3, etc, up to the last test of 98 with 99. Therefore there will always be at **least 99 comparisons**, no matter what. If the array was already sorted, then no swaps will occur and the sorting method will return after only one call to **floatLargestToTop()**, thus **99 comparisons and no swaps is possible**. However, if the array is unsorted before the call (as is usually the case) it will require more calls to **floatLargestToTop()**, so 99 is a lower bound on the possible number of comparisons -- there could be many more.



IncorrectQuestion 29

0 / 1 pts

Consider the class as partially defined here:

```
//class PizzaOrder -----
class PizzaOrder
{
public:
    static const int MAX_TOPPINGS = 20;
```

```
private:
    // instance members
    string toppings[MAX_TOPPINGS];
    int numToppings;

    // constructor
    public PizzaOrder();

    // accessor tells # toppings on pizza, i.e., # toppings in array
    int getNumToppings() { return numToppings; }

    // etc.
}

PizzaOrder::PizzaOrder()
{
    numToppings = 0;
}
```

We want to write an instance method, **addTopping()** that will take a **string** parameter, **topping**, and add it to the **toppings[]** array at the next available location as determined by the int member **numToppings**. Assume any **string** passed in is valid and no other method modifies the **toppings[]** array or **numToppings**.

The client would call it like so:

```
somePizzaOrder.addTopping( "onions" );
```

Which is a correct definition for **addTopping()** based on what you see, above?

☐

```
void PizzaOrder::addTopping( string topping )
{
    toppings[numToppings++] = topping;
}
```

☐

```
bool PizzaOrder::addTopping( string topping )
{
    if ( numToppings >= MAX_TOPPINGS )
        return false;
    toppings[numToppings++] = topping;
    return true;
}
```

☐

```
bool PizzaOrder::addTopping( string topping )
{
    if (numToppings > MAX_TOPPINGS)
        return false;
    toppings[numToppings++] = topping;
    return true;
}
```

☒

```
bool PizzaOrder::addTopping( string topping )
{
    if ( numToppings >= MAX_TOPPINGS || numToppings < 0 )
        return false;
    toppings[numToppings++] = topping;
    return true;
}
```

☐

```
bool PizzaOrder::addTopping( string topping )
{
    numToppings++;
    if ( numToppings >= MAX_TOPPINGS )
        return false;
    toppings[numToppings] = topping;
    return true;
}
```

☐

```
void PizzaOrder::addTopping( string topping )
{
    toppings[numToppings] = topping;
}
```

Feedback

We have to make sure that **numTopping** is not **>= MAX_TOPPING**, but besides that, other tests are not useful based on the information provided. We must increment **numTopping** each time a topping is successfully added. Incrementing **numToppings** should only happen if there is room to add a topping, otherwise the accessor will return an inaccurate # of toppings.



Question 30

1 / 1 pts

When an object is passed as a (i.e. non-reference or pointer) parameter to a method, modifying the members of that object from inside the method will result in a change to that object as seen from the client.

- ☐ True
- ☒ False

Feedback

In C++ objects are normally passed by value, and the entire object is sent into the method as a formal parameter, becoming a local variable. Changes to this local variable are not "felt" by the caller (client).



Question 31

1 / 1 pts

The statements

```
Card *card1, *card2, *card3, myCard;  
card1 = new Card;  
card2 = card1;
```

will cause how many card objects to be instantiated? (only one correct choice):

- ☒ 2
- ☐ 3
- ☐ none
- ☐ 4

Feedback

myCard is instantiated immediately, and a second **Card** object is instantiated in the **new** statement (but further pointers to that object do not cause additional instantiation).



Question 32

1 / 1 pts

A **stack** data structure, when implemented by a programmer as done in the modules as a LIFO data structure, has the following property (only one correct choice):

[Assume there are several items remaining in the stack when considering these options.]

- ☐ It allows the client to **pop ()** the earliest, i.e., first, **push ()** ed item (the **oldest** item in the stack).

- ☒ It allows the client to `pop()` the most recently `push()` ed item (the **newest** item in the stack).
- ☐ It allows the client to `pop()` **any** item off the stack, based on the parameter the client passes to `pop()`.

Feedback

Stacks, when implemented as a LIFO -- as we did -- only allow the newest item `push()` ed to be returned by `pop()`.



Question 33

1 / 1 pts

Here is an prototype for a working class **OldCellPhone** that has no errors (*you may see this exact class several times in this exam*):

```
class OldCellPhone
{
public:
    static const int MIN_CAP = 10;
    static const int MAX_CAP = 1000;
    static const string DEFAULT_DSCR;

private:
    string description;
    int memCapacity;
    bool camera;
    bool gps;
public:
    OldCellPhone(string dscr = DEFAULT_DSCR, int mem = MIN_CAP,
                 bool cam = false, bool gp = false);
    string toString();
    bool setMemCapacity(int mem);
    void setCamera( bool hasCam ) { camera = hasCam; }
    void setGps( bool hasGps ) { gps = hasGps; }
};
```

A programmer decides to make the static **DEFAULT_DSCR** *mutable*, removing its **const** status, as in:

```
static string DEFAULT_DSCR;
```

A **mutator** is added for this member. Any string whose length is at least 2 characters will be acceptable as a new **DEFAULT_DSCR**.

Check the true statements (there may be more than one correct answer):

If the mutator takes a **string** parameter, **newDefault**, a reasonable definition would be:



```
if ( newDefault.length() < 2 )
    return false;
DEFAULT_DSCR = newDefault;
return true;
```

If the mutator takes a **string** parameter, **newDefault**, a reasonable definition would be:



```
if ( newDefault.length() > 0 )
    return false;
DEFAULT_DSCR = newDefault;
return true;
```

☐ The mutator for this member will be an **instance method**.

If the mutator takes a **string** parameter, **newDefault**, a reasonable definition would be:



```
if ( newDefault.length() < 2 )
{
    this->newDefault = DEFAULT_DSCR;
    return false;
}
this->newDefault = newDefault;
return true;
```

☒ The mutator for this member will be a **static method**.

Feedback

It's a **static member** so its must must be a **static method**. The only mutator that tests the parameter and makes a reasonable assignment to the member **DEFAULT_DSCR** is the one code fragment marked.



IncorrectQuestion 34

0 / 1 pts

ClassA has a method, **void methodA(ClassA aObj)**, which takes, as a parameter, an object of the same **ClassA**.

There are no "sub-classes" (a topic from CS 2B we have not covered in CS 2A) **involved**. **ClassA** is the only class in the discussion and is not derived from any other user-defined classes.)

Check all the true statements. (Check *all* that apply):



methodA() can access private data of its **calling object** (the **this** object) directly, as in **somePrivateMember = something**, without the need for a public mutator or accessor and without the need to dereference anything.



If **methodA()** modifies a private instance member of its **calling object** (the **this** object), it will result in a simultaneous change of the corresponding private member in the parameter object, **aObj**, even if **aObj** is a different object than the calling object.



If **methodA()** modifies a private instance member of **the parameter object, aObj**, it will result in a simultaneous change of the corresponding private member in the **calling object** (the **this** object), even if **aObj** is a different object than the calling object.



methodA() can access private data of **aObj** directly, as in **aObj.somePrivateMember = something**, without the need for a public mutator or accessor.

Feedback

Member methods can access private data of objects of the same class. This applies to both the **calling object** as well as any objects that it may receive **as a parameter**. In the case of the calling object, no dereference is necessary. However, the calling object and the parameter object **may be different** -- and usually are: there is no need to pass a calling object as a parameter. The problem tells you to consider that they are distinct, in which case changes to one do not affect changes in the other.



Question 35

1 / 1 pts

A **static** member of some **ClassX** (check *all* that apply):



can, at any given time, store one value for **objA** and a **different** value for **objB**, two different objects of **ClassX**.



can be **public**.



can be modified from *any* client (like **Foothill's** `main()`) ***even if the client has not instantiated an object*** of **ClassX**. (You can assume that **ClassX** has some means for clients to modify the static member in question.)

☒ can have associated mutators and/or accessors.

☒ can be ***private***.

Feedback

One does not need objects to get at static data. A client can use the class name, not an object, as a technique to refer to the static member (if public) or a static accessor that indirectly modifies or accesses the static member.



IncorrectQuestion 36

0 / 1 pts

A static class method might take one or more objects of its class as parameters -- or it might take none -- depending on the purpose of the method.

☐ True

☒ False

Feedback

Static class methods may, or may not, take objects of the class as parameters, depending on their purpose.



Question 37

1 / 1 pts

Here is an prototype for a working class **OldCellPhone** that has no errors (*you may see this exact class several times in this exam*):

```
class OldCellPhone
{
public:
    static const int MIN_CAP = 10;
    static const int MAX_CAP = 1000;
    static const string DEFAULT_DSCR;

private:
    string description;
    int memCapacity;
    bool camera;
```

```

    bool    gps;
public:
    OldCellPhone(string dscr = DEFAULT_DSCR, int mem = MIN_CAP,
                  bool cam = false, bool gp = false);
    string toString();
    bool setMemCapacity(int mem);
    void setCamera( bool hasCam ) { camera = hasCam; }
    void setGps( bool hasGps ) { gps = hasGps; }
};

```

Two of the methods have a return type of **void**.

Check the true statements (there may be more than one):

☐

Even though their return types are **void**, either one of these method definitions can still return a **bool** as a functional return to the client.

☒

The **void** return types of these mutators does not pose a problem since it is impossible for a **bool** passed-in to be illegal; we don't have to test, and we don't have to return a **bool** response to the client, because if the client passes in a **bool**, it will naturally be either **true** or **false**, so no test is needed and the return can always be **void**.

☐

This is a mistake, because if the client passes an illegal **bool** argument down to either method, there is no way to report that fact back to the client via a return value (since their return types are **void**.)

☒

The client could call either method by passing the literal **false** as its argument.

☒

These methods are mutators, i.e, their name and intent is to modify private members **camera** and/or **gps**.

Hints

Feedback

A client passing a **bool** to the method can only pass the values **true** or **false**. Even if the client passes a compatible type (like an **int**) the meaning is still interpreted as zero = **false** and non-zero = **true**. There are no illegal values that can be passed, so the **void** return type is compatible and appropriate for this mutator method and is no mistake. As a **void** method, it cannot return any value, including **bool**.



Question 38

1 / 1 pts

Within an instance method, a programmer might utilize the special syntax `this->someMember` to reasonably assist with which of the following challenges? (Check *all* that apply):

☐ To get at private data of a **parameter** that is passed to the method.



To get at private data of the calling object, since without `this->` before the member, that member's **privacy prohibits direct access**.



To get at private data of the **calling object** (the **this** object), if there is a **local variable that has the same name** as the instance member in question.



To enhance readability, if the programmer wants to allow the reader to easily distinguish instance members in their methods from local variables of those methods. While not required, prepending **this->** to each member used in the method can only mean that those variables refer to the calling object's members.

Feedback

`this->` is of no help in getting around privacy or accessing a parameter's members. There is also no problem based on privacy with getting at the calling object's members. However, disambiguating members from local variables, either due to local variable "hiding" or readability is a reasonable use of the `this->` syntax.



Question 39

1 / 1 pts

Consider a **binary search algorithm** as described in the modules. Assume that the array to be searched is pre-sorted, so that sort is **not** part of the search algorithm.

Searching an array using a **binary search** (check *all* choices that apply):

☐ ... is usually slower for each search than a simple linear search.

☒ ... is usually faster for each search than a simple linear search.

☒ ... requires more code and logic than a simple linear search.

☒ ... requires a pre-sorted array in order to work.

☐ ... does a sort as part of the search algorithm.

Feedback

A binary search takes less time for each search than a simple linear search, but requires that the array be pre-sorted (an operation that we presumed to be done prior to the search, and was not part of the search, as stated in the question.) The algorithm is much more complex than a linear search, requiring a recursive method call.



Question 40

1 / 1 pts

A static class method can be accessed without referring to any objects of the class.

- ☒ True
- ☐ False

Feedback

Static class methods (and variables) can and should be accessed by dereferencing the class name, not an object of the class. No objects are needed.

Quiz Score: 35.33 out of 40